

Laboratorio 03

Búsqueda de claves AES con Open MPI

Universidad del Valle de Guatemala

Computación Paralela y Distribuida

Carlos Daniel Estrada Vega · Carné 20853

Trabajo individual · 5 de octubre de 2026

Objetivo

Analizar el programa secuencial de búsqueda de claves AES, corregir sus limitaciones y desarrollar una versión paralela. Comparé ambas versiones con el mismo archivo cifrado y medí su rendimiento con dos, tres y cuatro procesos.

Qué es AES y dónde se utiliza

AES es un algoritmo de cifrado simétrico: la misma clave permite cifrar y descifrar. AES-128 utiliza una clave de 128 bits, bloques de 128 bits y diez rondas de transformación. [1]

Campo	Aplicación
Comunicaciones web	TLS 1.3 incluye AES-GCM para proteger el tráfico de conexiones HTTPS. [2]
Redes privadas virtuales	IPsec ESP puede utilizar AES-GCM para proteger y autenticar paquetes. [3]
Almacenamiento	XTS-AES se utiliza para proteger la confidencialidad de datos en dispositivos de almacenamiento. [4]

Transformaciones principales

Transformación	Función
SubBytes	Sustituye cada byte mediante una tabla no lineal.
ShiftRows	Desplaza las filas del estado.
MixColumns	Mezcla los bytes de cada columna.
AddRoundKey	Combina el estado y la clave de ronda mediante XOR.

El estado organiza los 16 bytes en una matriz de 4×4 . La expansión de clave genera las claves de ronda. Para mensajes completos también se necesita elegir un modo de operación y resolver cómo tratar la longitud del texto. [1]

Cifrado y descifrado de AES 128

El diagrama muestra el procesamiento de un bloque. Las claves de ronda alimentan cada operación AddRoundKey; el descifrado las utiliza en orden inverso. La ronda final omite MixColumns y, en el sentido inverso, InvMixColumns. [1]

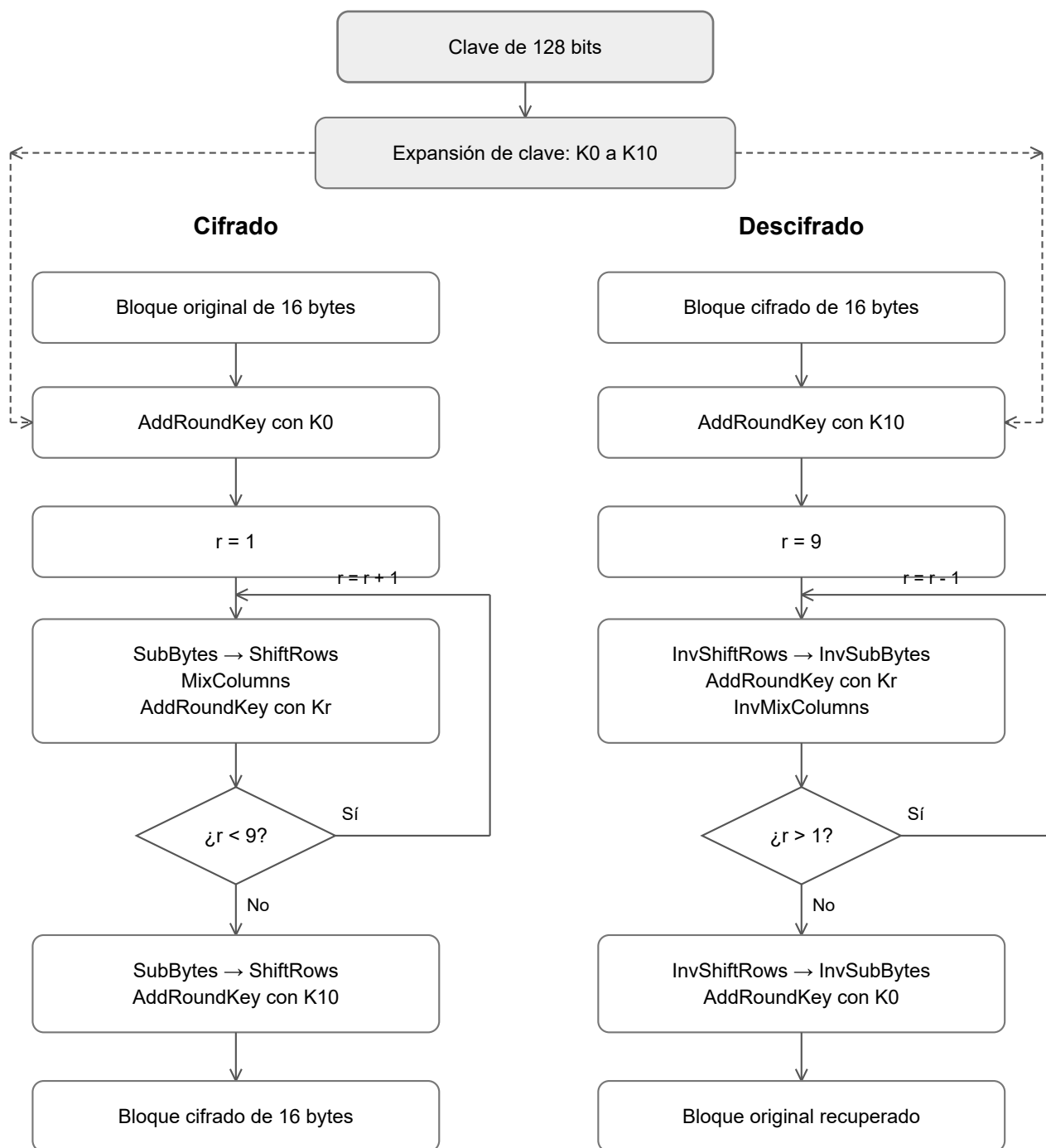


Figura 1. Flujo del algoritmo AES-128. Las flechas discontinuas muestran la intervención de la expansión de clave.

En el programa mejorado se usa GCM: AES cifra bloques de contador y el resultado se combina con el mensaje. GCM admite longitudes que no son múltiplos de 16 y añade una etiqueta de autenticación. Por eso la operación de descifrado GCM usa internamente el cifrado AES de los contadores. [5]

Análisis del programa original

Uso correcto de AES 128

El programa utiliza **EVP_aes_128_ecb()** y entrega una clave de 16 bytes. El texto “Puedes lograrlo!” ocupa exactamente 16 bytes; su terminador no se cifra. Desactivar el relleno es correcto para ese bloque. Las operaciones de cifrado y descifrado se realizan con EVP y se comprueban sus resultados. No encontré un error en la selección de AES-128.

Construcción de la clave

La función **make_key()** deja ocho bytes en cero y escribe la candidata en los ocho restantes. La clave 12345 se representa como **000000000000000000000000003039**. Su tamaño es válido para AES-128, pero su construcción es predecible. Dentro del rango del ejercicio solamente varían 20 bits.

Modo ECB

ECB procesa los bloques por separado. Con una misma clave, bloques originales iguales producen bloques cifrados iguales; además, no detecta modificaciones por sí solo. En este ejemplo hay un único bloque, así que no aparecen patrones entre varios bloques, pero el modo limita la extensión del programa a mensajes reales.

Verificación mediante texto conocido

La búsqueda acepta una candidata cuando **memcmp()** confirma que el texto descifrado coincide con el original. Esto es válido para un experimento con texto conocido. Su limitación es que exige disponer del mensaje esperado. **SECRET_KEY** se usa para preparar el cifrado; el ciclo de búsqueda no consulta directamente ese valor.

Espacio de claves y trabajo efectivo

El rango contiene 2^{20} candidatas: 1 048 576. AES-128 tiene 2^{128} claves posibles, aproximadamente $3,40 \times 10^{38}$. El laboratorio explora una fracción 2^{-108} de ese espacio. Encontrar una clave construida de esta forma no significa romper AES-128.

La búsqueda empieza en cero y se detiene en 12345, por lo que prueba 12 346 candidatas. En la evaluación automatizada recuperó el mensaje en **0,011494 s**; en la captura adicional, en **0,009547 s**. No se compara directamente con la versión mejorada: cambian el modo criptográfico y la cantidad de trabajo.

Medición y otras limitaciones

El programa ya imprime el tiempo obtenido con **CLOCK_MONOTONIC**, después de preparar el cifrado. También reutiliza el contexto de OpenSSL. Sus principales restricciones son el mensaje fijo, la clave fija y el rango definido al compilar. La versión original se conserva sin cambios para documentar el punto de partida.

Referencia al código original: **make_key**, líneas 46-55; **crypt_block**, 62-100; preparación y búsqueda, 136-154; salida del tiempo, 165-166.

Mejoras y distribución del trabajo

Autor de todas las mejoras: Carlos Daniel Estrada Vega.

Mejoras implementadas

1. Autenticación. Reemplacé ECB por AES-128-GCM. Se genera un nonce aleatorio de 12 bytes y una etiqueta de 16 bytes. Una candidata se acepta únicamente si OpenSSL valida la etiqueta; el texto provisional se descarta cuando falla. Esto permite verificar sin guardar el texto original. [5, 6]

2. Preparación separada. El comando **preparar** crea un archivo que contiene versión, longitud, nonce, etiqueta y texto cifrado. El comando **buscar** solo recibe ese archivo y el límite. Ni la clave secreta ni el mensaje original se guardan en el archivo.

3. Flexibilidad. El mensaje se recibe como argumento, de 1 a 4096 bytes. La clave de prueba, el rango y el tamaño de lote MPI también son configurables. Se rechazan números negativos, entradas inválidas y archivos mal formados. El límite de búsqueda permitido es de 1 a 2^{32} candidatas.

4. Comparación reproducible. Ambas versiones comparten las rutinas de AES y leen exactamente el mismo archivo. Se conservó la reutilización del contexto EVP y se añadió el contador de candidatas. La asignación numérica de claves sigue siendo educativa: GCM no convierte ese conjunto predecible en claves seguras para producción. Un nonce no debe reutilizarse con la misma clave. [5]

Versión con Open MPI

El proceso 0 lee el archivo y distribuye los datos con **MPI_Bcast**. Cada proceso tiene su propio contexto EVP y prueba lotes de candidatas. Para la ronda r , el proceso p de un grupo de n procesos recibe:

Inicio = $(r \times n + p) \times B$

Fin exclusivo = $\text{mínimo}(\text{Inicio} + B, \text{límite})$

Ronda	Proceso 0	Proceso 1	Proceso 2
0	0 a 3	4 a 7	8 a 11
1	12 a 15	16	Sin candidatas

Ejemplo: límite 17, tres procesos y lote $B = 4$. Los intervalos son disjuntos y cubren todas las candidatas al agotar el rango.

Al terminar cada lote, **MPI_Allreduce** con **MPI_MIN** comunica la clave encontrada; **UINT64_MAX** indica que no hubo coincidencia. Todos los procesos participan, incluso quienes no tienen candidatas en la última ronda. Si alguien encuentra la clave, los demás terminan su lote actual y todos salen del ciclo. Si el rango se agota, finalizan sin resultado. Así se evita que un proceso salga antes de una operación colectiva. [7]

Resultados de rendimiento

Entorno: Ubuntu 26.04.1 LTS en WSL2, Intel Core Ultra 5 125U, 14 CPU lógicas visibles y 15 GiB de memoria. GCC 15.2.0, Open MPI 5.0.10 y OpenSSL 3.5.5. Compilación con C11, -O2, -Wall, -Wextra y -Wpedantic.

Usé el mensaje “Puedes lograrlo!”, el rango [0, 1 048 576), la clave 1 048 575 y lotes de 4096. En todas las ejecuciones se probaron **1 048 576 candidatas** y se recuperaron la misma clave y el mismo mensaje. La preparación del archivo y su lectura quedan fuera del tiempo de búsqueda.

Se hizo un calentamiento por configuración y cinco repeticiones medidas, alternando el orden. En MPI, la medición comienza después de MPI_Barrier e incluye la búsqueda y la coordinación de parada. Se reporta el mayor tiempo entre procesos. No incluye el arranque de mpirun, la impresión ni las reducciones finales de estadísticas.

Tiempos medidos en segundos

Repetición	Secuencial	2 procesos	3 procesos	4 procesos
1	0.817179	0.602741	0.580707	0.375598
2	1.159431	0.638197	0.620884	0.391320
3	1.160304	0.471559	0.565751	0.360044
4	1.095423	0.569207	0.597680	0.352588
5	1.193237	0.539759	0.608436	0.399767

Speedup(n) = $T_{\text{secuencial}} / T_{\text{paralelo},n}$. La siguiente tabla utiliza las medianas de los tiempos; eficiencia = speedup / n.

Procesos	Mediana (s)	Speedup	Eficiencia
1	1.159431	1.000	100.00%
2	0.569207	2.037	101.85%
3	0.597680	1.940	64.66%
4	0.375598	3.087	77.17%

Speedup de cada repetición

Repetición	2 procesos	3 procesos	4 procesos
1	1.356	1.407	2.176
2	1.817	1.867	2.963
3	2.461	2.051	3.223
4	1.924	1.833	3.107
5	2.211	1.961	2.985

Cuatro procesos dieron el mejor resultado: **3,087 veces** el rendimiento secuencial, equivalente a una reducción aproximada del 67,6% del tiempo mediano. Tres procesos no mejoraron respecto de dos. La variación entre repeticiones, la planificación de WSL y los costos de sincronización pueden explicar este comportamiento. El speedup ligeramente superior a dos con dos procesos no demuestra escalamiento superlineal sostenido.

Capturas de compilación y ejecución

Capturas tomadas en Ubuntu desde Windows Terminal, antes de reorganizar las carpetas del repositorio. Esta sesión adicional verifica la compilación y los resultados; las cinco repeticiones de la evaluación se conservan en la tabla de rendimiento y en los registros originales.

Compilación de los tres programas sin advertencias

```
This message is shown once a day. To disable it please create the
/home/carlosestrada/.hushlogin file.
carlostrada@GP01-CDESTRADA:~$ cd /mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas
make -B
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_secuencial.c -o busqueda_clave_aes_secuencial -lcrypto
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_mejorada.c -o busqueda_clave_aes_mejorada -lcrypto
mpicc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_mpi.c -o busqueda_clave_aes_mpi -lcrypto
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$
```

Programa secuencial original

```
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$ ./busqueda_clave_aes_secuencial
Clave encontrada: 12345
Mensaje: Puedes lograrlo!
Ejecucion: secuencial
Tiempo: 0.009547 segundos
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$
```

Programa secuencial mejorado

```
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$ ./busqueda_clave_aes_mejorada buscar ../res
ultados/datos.aes 1048576
Ejecucion: secuencial mejorada | Procesos: 1
Candidatas probadas: 1048576
Clave encontrada: 1048575
Clave AES (hex): 00000000000000000000000000000000ffff
Mensaje: Puedes lograrlo!
Autenticacion GCM: correcta
Tiempo de busqueda: 1.056688770 segundos
```

El original recupera 12345. La versión mejorada prueba 1 048 576 candidatas y recupera 1048575, con el mensaje esperado y autenticación correcta.

Evidencia de ejecución con Open MPI

Las tres ejecuciones recuperaron la clave 1048575 y el mensaje **Puedes lograrlo!**, con autenticación correcta.

Dos procesos

```
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$ mpirun --allow-run-as-root --bind-to core -
n 2 ./busqueda_clave_aes_mpi ../resultados/datos.aes 1048576 4096
Ejecucion: Open MPI | Procesos: 2
Candidatas probadas: 1048576
Clave encontrada: 1048575
Clave AES (hex): 00000000000000000000000000000000ffff
Mensaje: Puedes lograrlo!
Autenticacion GCM: correcta
Tiempo de busqueda: 0.574145790 segundos
```

Tres procesos

```
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$ mpirun --allow-run-as-root --bind-to core -
n 3 ./busqueda_clave_aes_mpi ../resultados/datos.aes 1048576 4096
Ejecucion: Open MPI | Procesos: 3
Candidatas probadas: 1048576
Clave encontrada: 1048575
Clave AES (hex): 00000000000000000000000000000000ffff
Mensaje: Puedes lograrlo!
Autenticacion GCM: correcta
Tiempo de busqueda: 0.541838446 segundos
```

Cuatro procesos

```
carlostrada@GP01-CDESTRADA:/mnt/c/Users/carlos.estrada/Documents/U/Laboratorio-3---Paralela/entrega/programas$ mpirun --allow-run-as-root --bind-to core -
n 4 ./busqueda_clave_aes_mpi ../resultados/datos.aes 1048576 4096
Ejecucion: Open MPI | Procesos: 4
Candidatas probadas: 1048576
Clave encontrada: 1048575
Clave AES (hex): 00000000000000000000000000000000ffff
Mensaje: Puedes lograrlo!
Autenticacion GCM: correcta
Tiempo de busqueda: 0.516579520 segundos
```

Procesos	Tiempo de la captura (s)	Speedup de esta sesión
2	0.574145790	1.840
3	0.541838446	1.950
4	0.516579520	2.046

Base de estos speedups: secuencial mejorada de la captura, 1,056688770 s. Esta sesión se presenta por separado de las medianas de la evaluación automatizada.

Verificación y archivos entregados

El script **evaluar.py** terminó con **17 comprobaciones funcionales aprobadas**, además de validar las 20 ejecuciones medidas. El registro completo se conserva en **07_pruebas_completas.txt**.

Prueba	Resultado
Vector de respuesta conocida AES-128-GCM	Cifrado, etiqueta y descifrado coinciden con el vector.
Claves al inicio, en medio y al final	Misma clave y mensaje en secuencial y MPI.
Reparto irregular y procesos sin trabajo	Finalización correcta con rangos no divisibles y más procesos que candidatas.
Rango agotado	Se prueban exactamente 17 candidatas; salida 2.
Etiqueta alterada	Ninguna versión acepta el mensaje como válido.
Longitudes de 1 y 4096 bytes	Recuperación y autenticación correctas.
Entradas inválidas y archivo inexistente	Salida de error controlada; MPI termina sin bloqueo.

Contenido de la entrega

Informe.pdf: informe del laboratorio, en la raíz del repositorio.

programas: secuencial original, secuencial mejorado, versión MPI, cabecera compartida, Makefile, test_aes.c y evaluar.py.

resultados: datos.aes, tiempos.csv, resumen.json y tiempos_capturas.csv.

evidencias: seis capturas; los registros completos están en evidencias/registros.

README.md: instrucciones para compilar, ejecutar y repetir las pruebas.

Referencias

[1] NIST. FIPS 197, Advanced Encryption Standard, actualización 2023.

<https://csrc.nist.gov/pubs/fips/197/final>

[2] IETF. RFC 8446, TLS 1.3, 2018; sección 9.1.

<https://www.rfc-editor.org/rfc/rfc8446.html#section-9.1>

[3] IETF. RFC 4106, AES-GCM en IPsec ESP, 2005.

<https://www.rfc-editor.org/rfc/rfc4106.html>

[4] NIST. SP 800-38E, XTS-AES para dispositivos de almacenamiento, 2010.

<https://csrc.nist.gov/pubs/sp/800/38/e/final>

[5] NIST. SP 800-38D, GCM y GMAC, 2007.

<https://csrc.nist.gov/pubs/sp/800/38/d/final>

[6] OpenSSL. EVP_EncryptInit, operaciones de cifrado autenticado.

https://docs.openssl.org/3.1/man3/EVP_EncryptInit/

[7] Open MPI. MPI_Allreduce, manual de la API.

https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Allreduce.3.html

Fuentes consultadas el 5 de octubre de 2026. El programa original y la consigna fueron proporcionados como material del laboratorio.